

# DSS5201 Data Visualisation

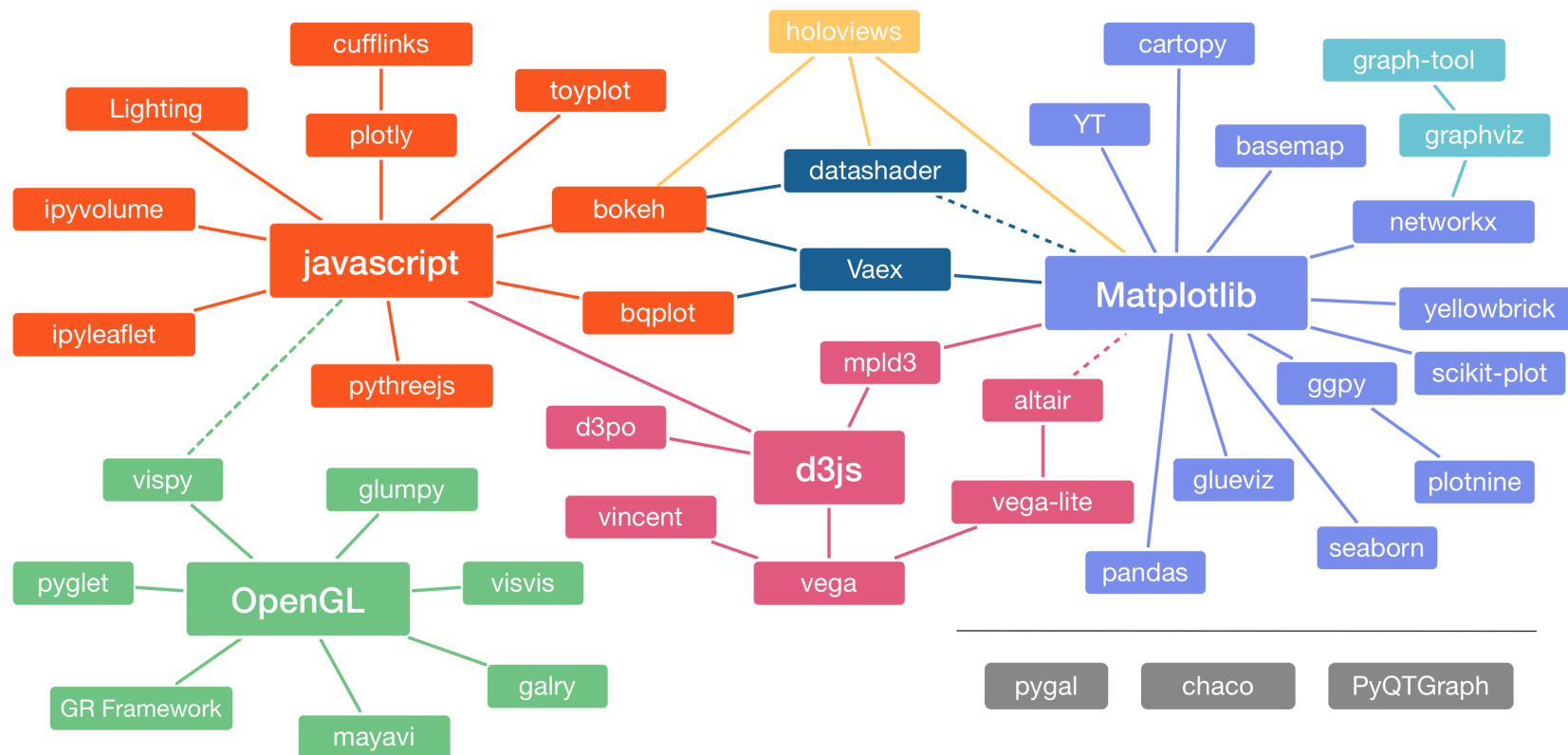
Week 12 Interactive Visualisation with plotly

David CHEW

Semester 2 AY25/26

# Introduction

## plotly for Interactive Visualisation



plotly is a library for interactive graphs.

We first import the libraries we need —

```
1 import pandas as pd
2 import plotly.express as px
```

# Using plotly in Jupyter Notebook

plotly generates interactive visualisations in Jupyter Notebooks.

- By default, these plots **will not appear** in the exported HTML output. 
- To ensure they are visible and interactive, we need to manually set the rendering mode.

```
1 import plotly.io as pio
2 pio.renderers.default = 'notebook_connected'
```



# Selected plotly Functions

# Selected plotly Functions

Let us explore the following `plotly` functions:

- `px.histogram()`
- `px.box()`
- `px.density_heatmap()`
- `px.scatter()`
- `px.scatter_3d()`
- `px.choropleth()`

# Penguins Data

Let us use the Penguins data set to demonstrate the above functions.

```
1 penguins = pd.read_csv('../data/penguins.csv').dropna(subset=['sex'])
2 penguins.head()
```

|   | species   | island | bill_length_mm | bill_depth_mm | flipper_length_mm | body_mass_g | sex    | year |
|---|-----------|--------|----------------|---------------|-------------------|-------------|--------|------|
| 0 | Chinstrap | Dream  | 46.9           | 16.6          | 192               | 2700        | female | 2008 |
| 1 | Adelie    | Biscoe | 36.5           | 16.6          | 181               | 2850        | female | 2008 |
| 2 | Adelie    | Biscoe | 36.4           | 17.1          | 184               | 2850        | female | 2008 |
| 3 | Adelie    | Biscoe | 34.5           | 18.1          | 187               | 2900        | female | 2008 |
| 4 | Adelie    | Dream  | 33.1           | 16.1          | 178               | 2900        | female | 2008 |

# Penguins Data (with plotly ...)

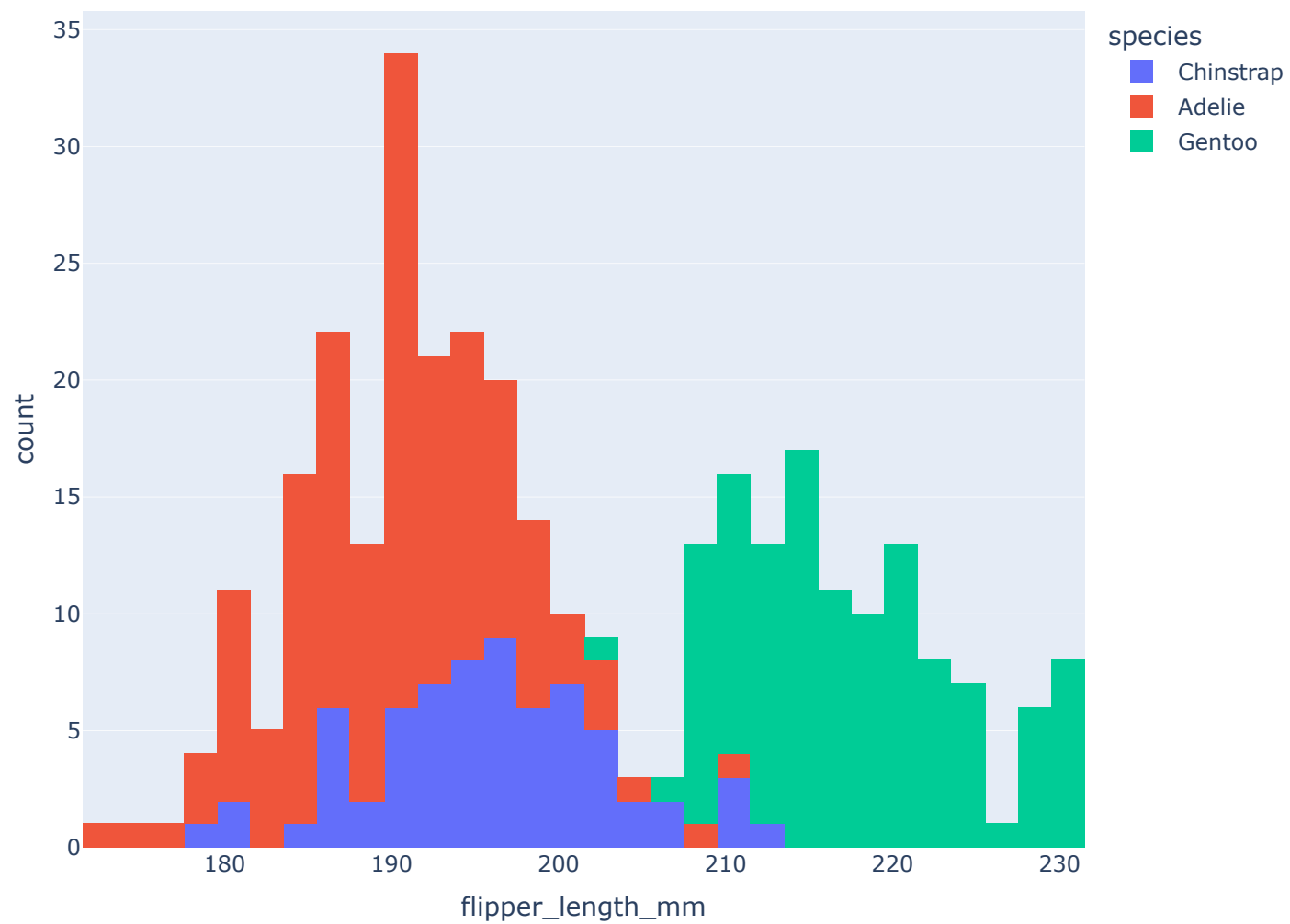
## Histogram

`px.histogram()` creates an interactive histogram.

```
1 fig = px.histogram(penguins, x = "flipper_length_mm",  
2                     color = "species", nbins = 30,  
3                     title = "Distribution of Penguin Flipper Lengths")  
4 fig.show()
```



Distribution of Penguin Flipper Lengths



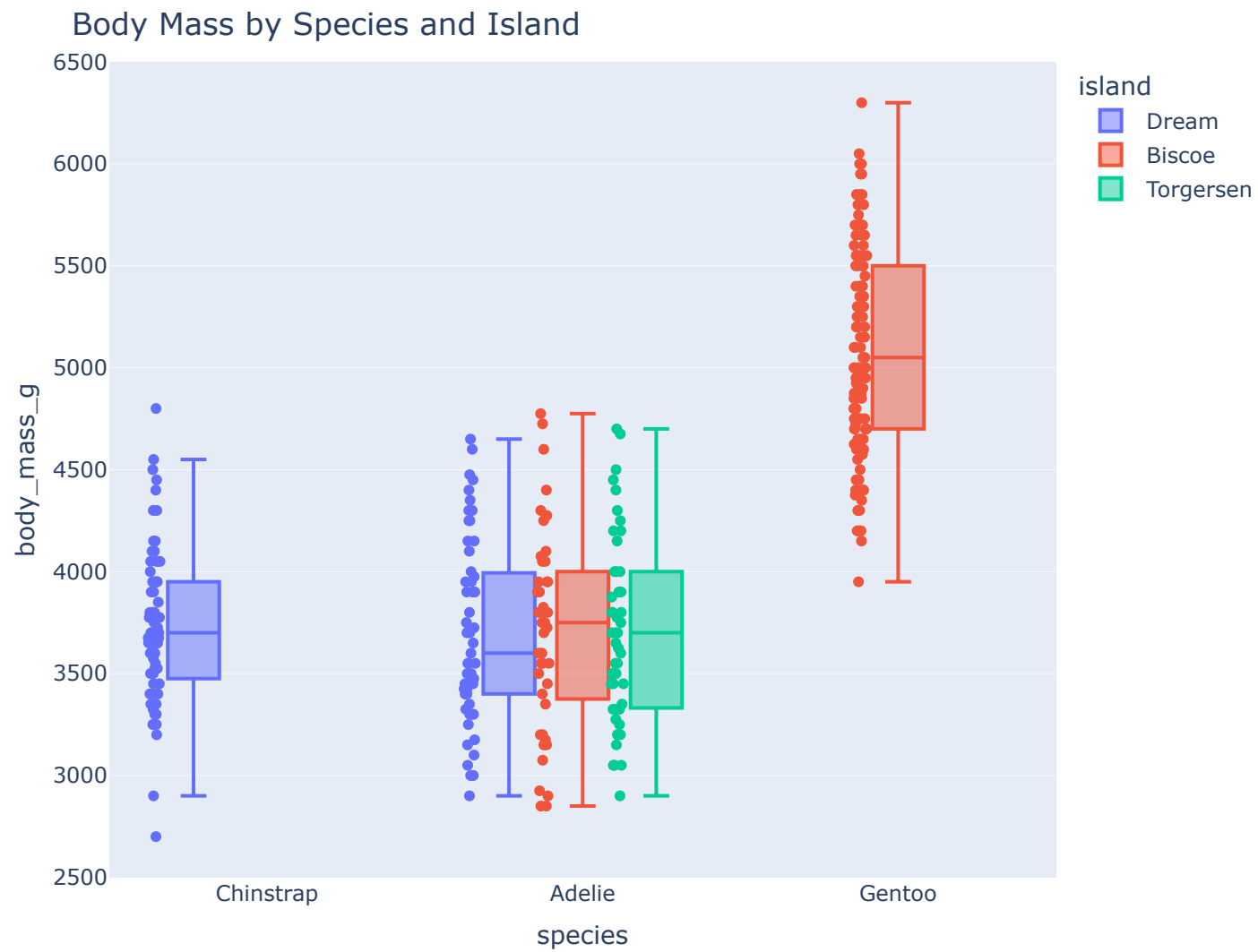
# Penguins Data (with plotly ...)

## Box Plot

`px.box()` creates an interactive box plot.

```
1 fig = px.box(penguins, x = "species", y = "body_mass_g", color = "island",  
2             points="all", title = "Body Mass by Species and Island")  
3 fig.show()
```





# Penguins Data (with plotly ...)

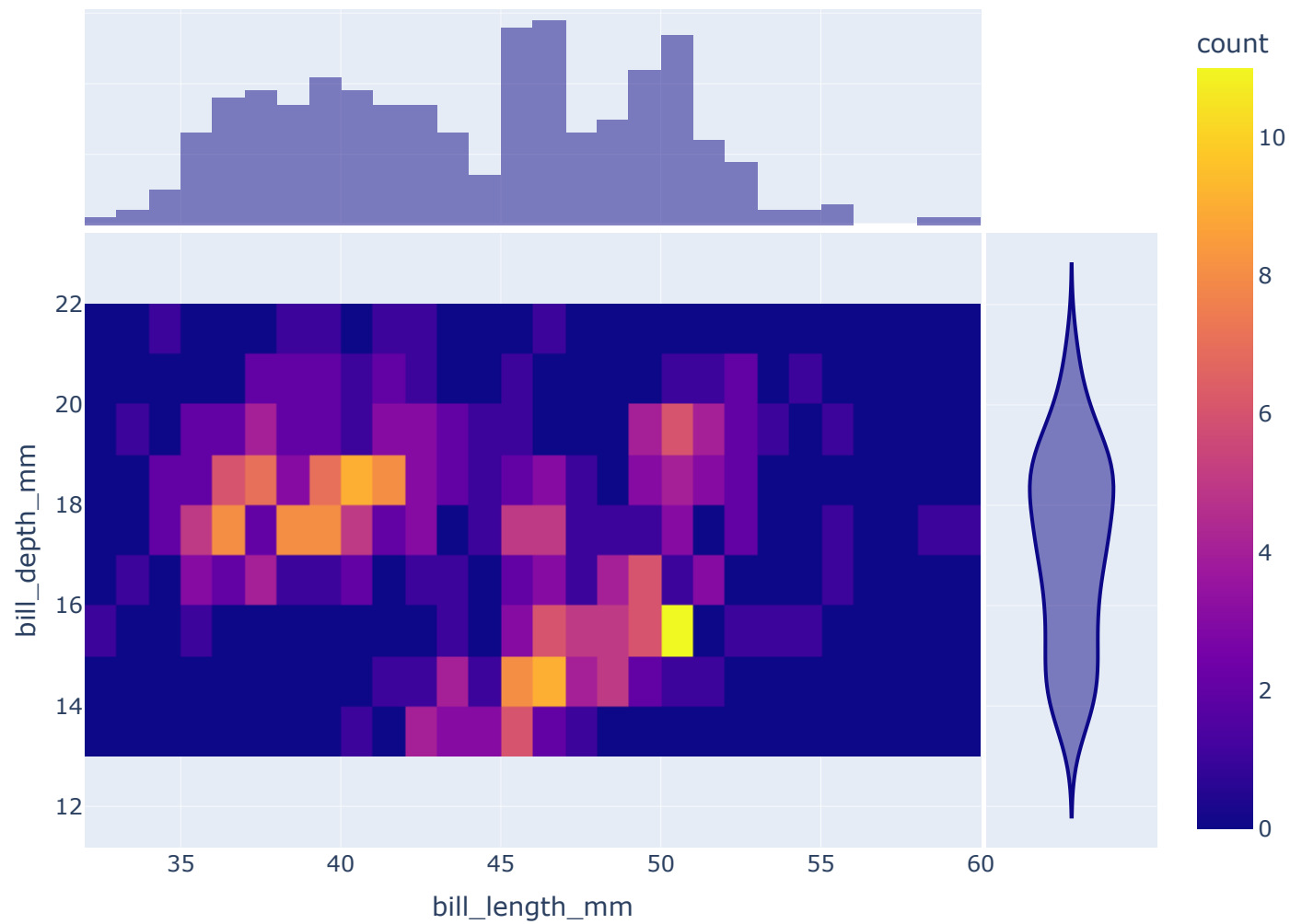
## Heatmap

`px.density_heatmap()` creates an interactive heatmap.

```
1 fig = px.density_heatmap(penguins,  
2                           x = "bill_length_mm", y = "bill_depth_mm",  
3                           marginal_x = "histogram", marginal_y = "violin",  
4                           nbinsx = 30,  
5                           title = "Bill Length vs Bill Depth Density")  
6 fig.show()
```

Here are the built-in colour scales in plotly.

Bill Length vs Bill Depth Density



# Penguins Data (with plotly ...)

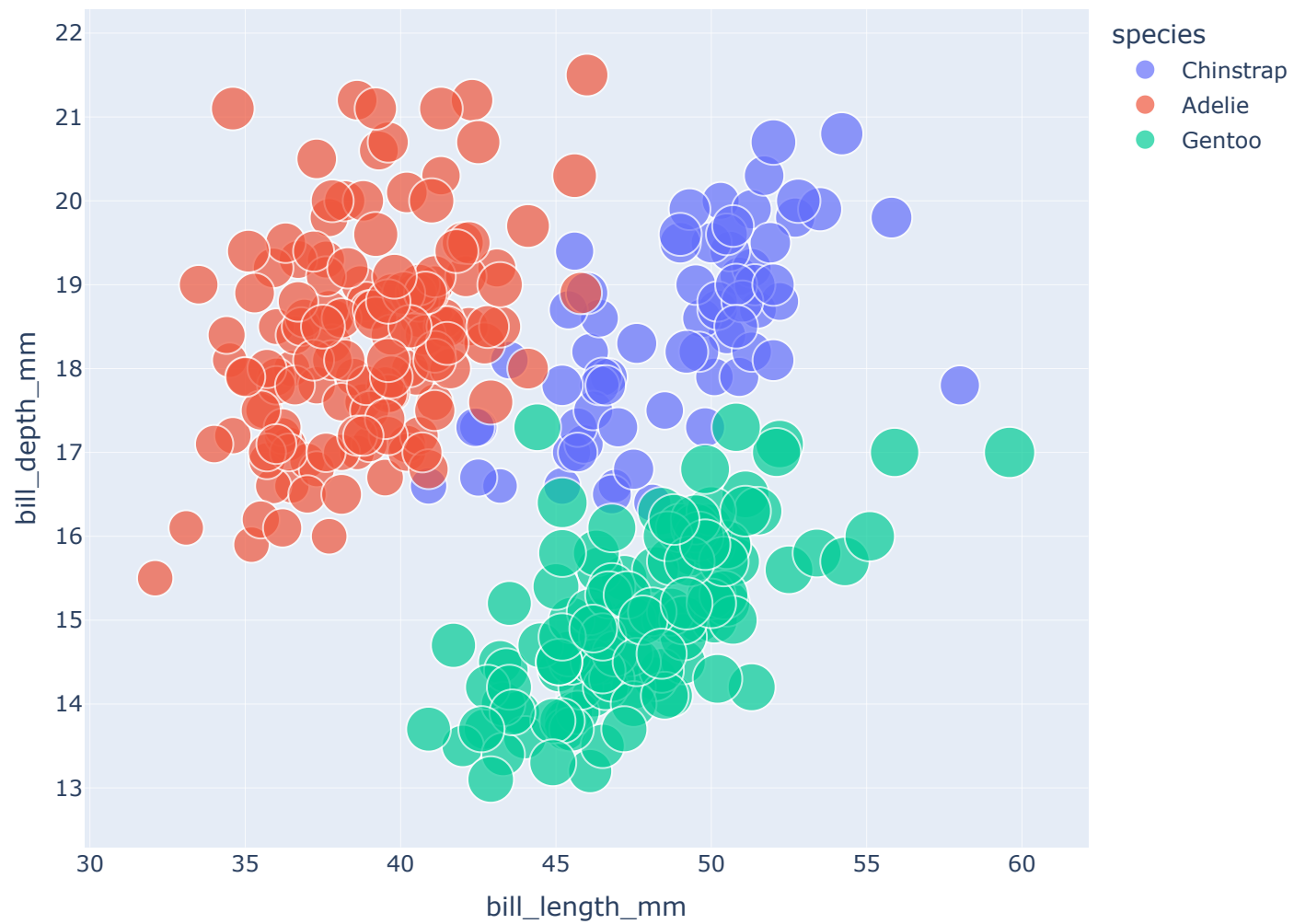
## Scatter Plot

`px.scatter()` creates an interactive scatter plot.

```
1 fig = px.scatter(penguins, x = "bill_length_mm", y = "bill_depth_mm",  
2                   color = "species", size = "body_mass_g",  
3                   hover_name = "island",  
4                   title = "Penguin Bill Size and Body Mass")  
5 fig.show()
```



Penguin Bill Size and Body Mass



# Penguins Data (with plotly ...)

## 3D Scatter Plot

`px.scatter_3d()` creates an interactive 3D scatter plot.

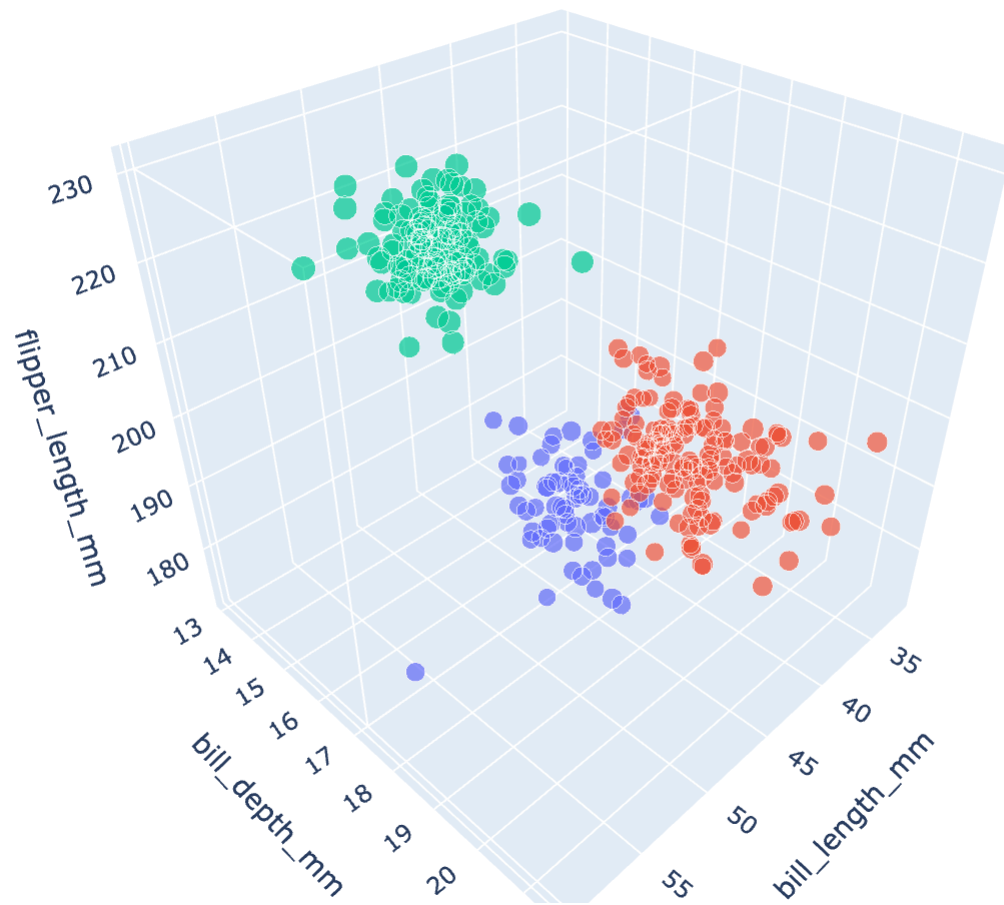
```
1 fig = px.scatter_3d(penguins, x = "bill_length_mm",  
2                       y = "bill_depth_mm", z = "flipper_length_mm",  
3                       color = "species", size = "body_mass_g",  
4                       title = "3D View of Penguin Measurements")  
5 fig.show()
```



3D View of Penguin Measurements

species

- Chinstrap
- Adelie
- Gentoo



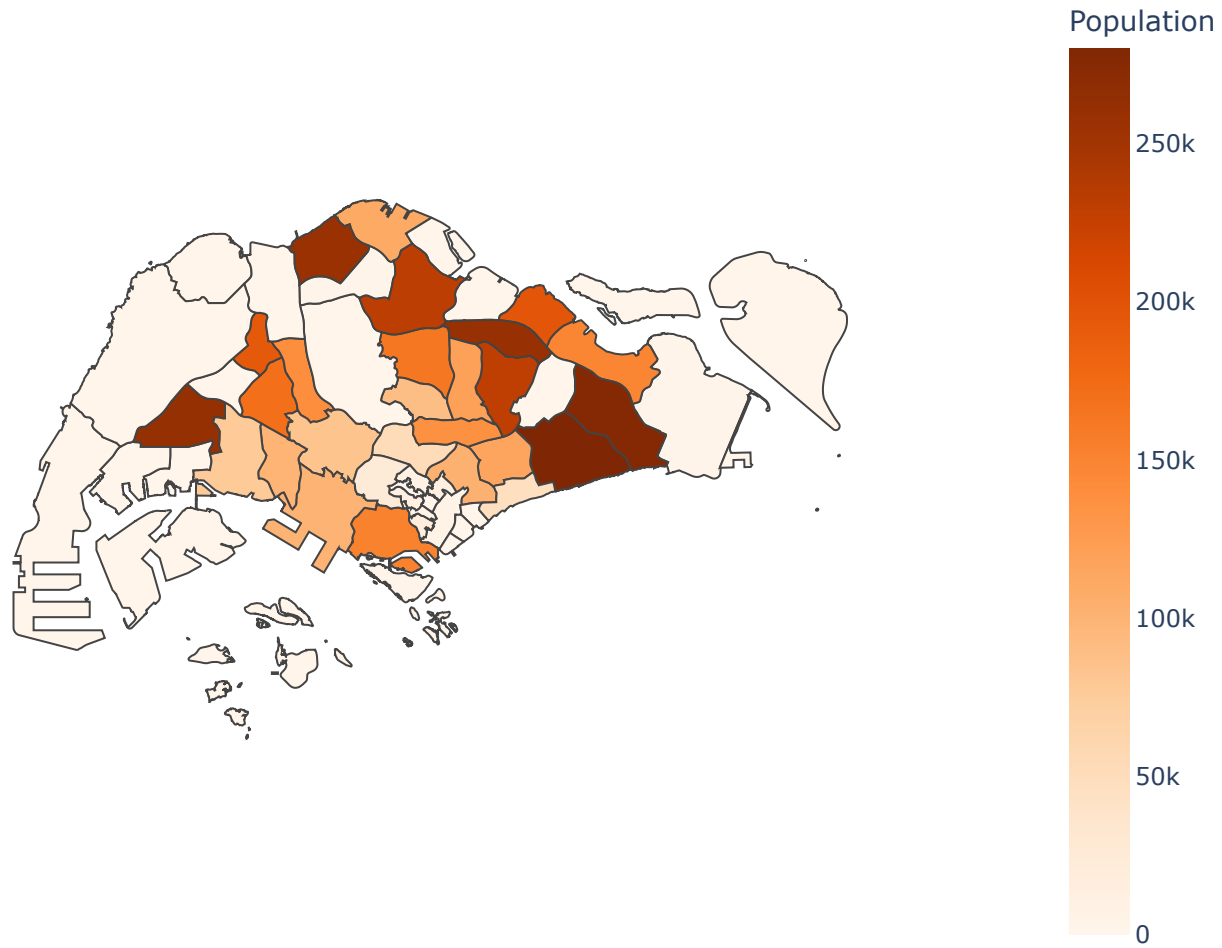
# Singapore Map (with plotly ...)

## Choropleth Map

We create an interactive version of the Singapore map using `px.choropleth()`.

```
1 import geopandas as gpd
2 sg = gpd.read_file('../data/wk12_MasterPlan2019.geojson')
3 sg_pop = pd.read_csv('../data/wk12_sg_pop.csv')
4 sg_pop = sg_pop[sg_pop['Time'] == 2023]
5
6 fig = px.choropleth(
7     sg_pop,                                # data source
8     geojson = sg.__geo_interface__,        # spatial shapes
9     locations = 'PA',                      # 'key' from csv
10    featureidkey = "properties.town",      # 'key' from geojson
11    color = 'total',
12    color_continuous_scale = "Oranges",
13    title = "Singapore's Demographic Distribution in 2023",
14    hover_name = 'PA',
15    labels = {'total': 'Population'})
16
17 # zoom map & adjust layout
18 fig.update_geos(fitbounds = "locations", visible = False)
19 fig.update_layout(margin = {"r":0,"t":40,"l":0,"b":0})
20 fig.show()
```

## Singapore's Demographic Distribution in 2023



# Nobel Prizes Data (with plotly ...)

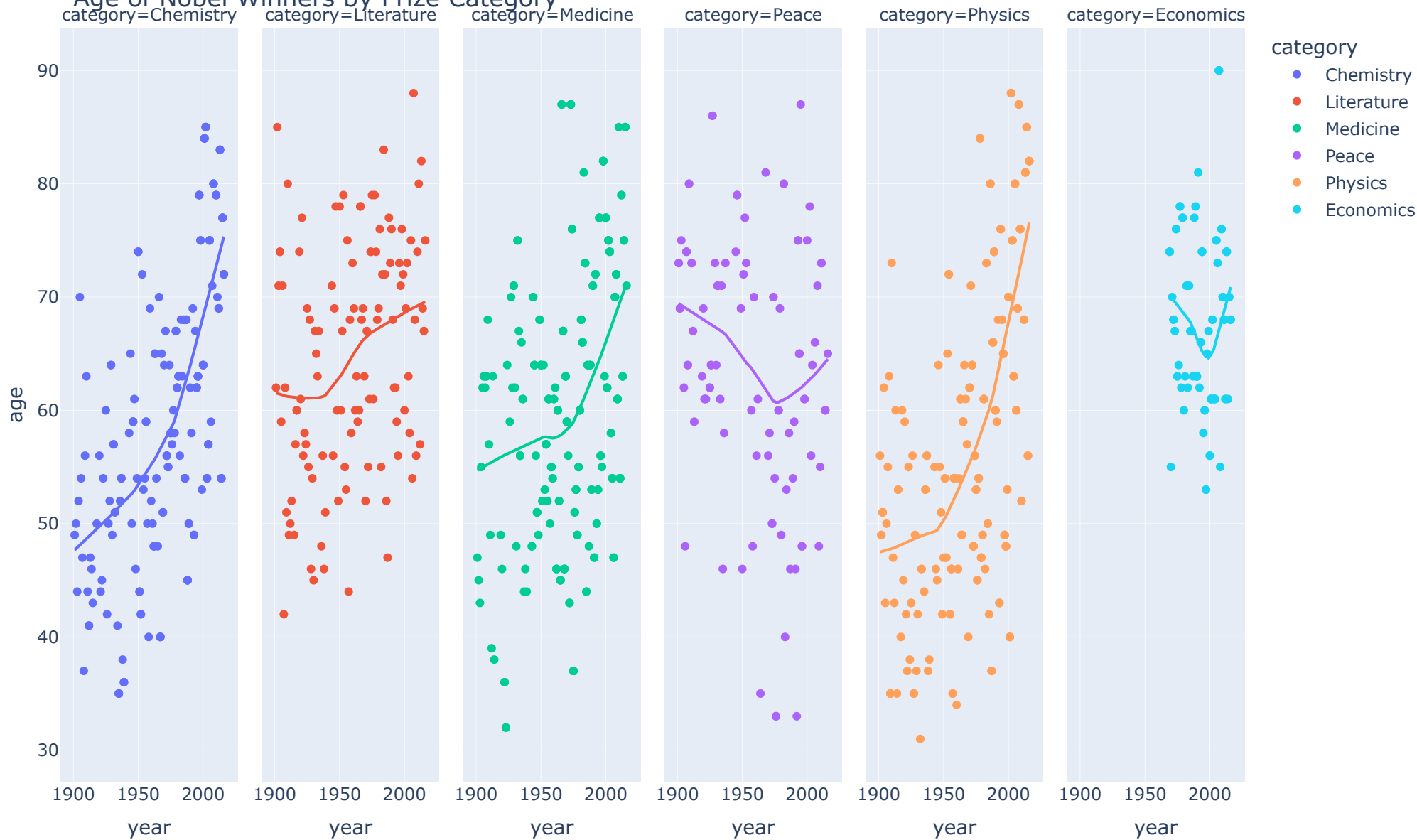
## Age of Winners Across Fields

Next, we work with the age distribution of Nobel prize winners.

- We will facet the plot by prize category and assign colors across subplots.
- For `trendline`, we can add the LOWESS smoother.

```
1 df = pd.read_csv('../data/wk10_nobel_prizes16.csv')
2 nobel = df[df['gender'].isin(['Female', 'Male'])]
3 fig = px.scatter(nobel, x = 'year', y = 'age', width = 1000, height = 600,
4                 title = 'Age of Nobel Winners by Prize Category',
5                 hover_name = "fullName", color = 'category',
6                 facet_col = 'category', trendline = 'lowess')
7 fig.show()
```

# Age of Nobel Winners by Prize Category



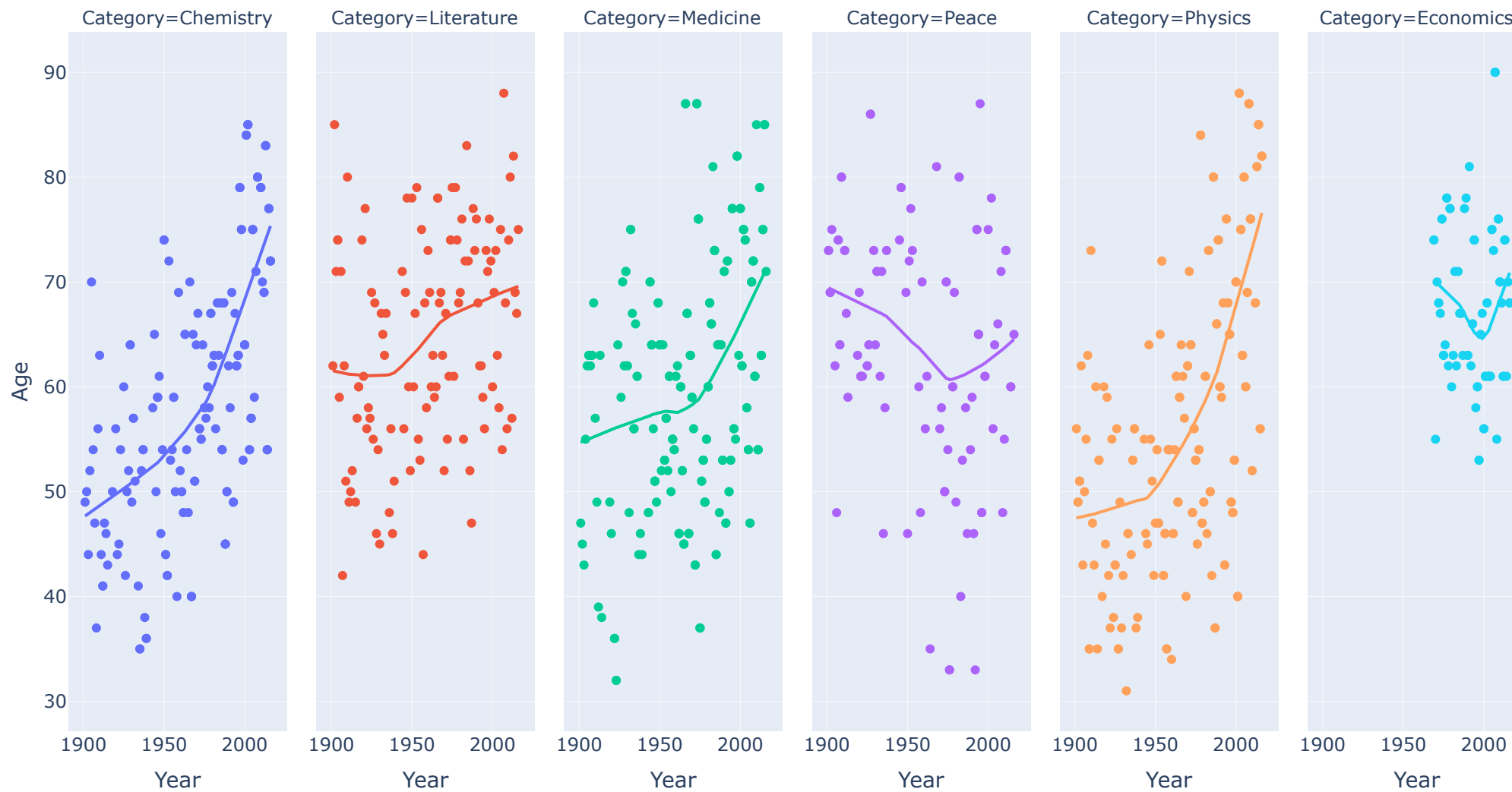
# Nobel Prizes Data (with plotly ...)

## Age of Winners Across Fields: Title, Labels, and Legend

- We can use `update_layout()` to adjust the appearance and layout.
- Since the names of categories have been displayed for each subplot, we can remove the color legend box.

```
1 fig = px.scatter(nobel, x = 'year', y = 'age', width = 1000, height = 600,  
2                   title = 'Age of Nobel Winners by Prize Category',  
3                   hover_name = "fullName", color = 'category',  
4                   facet_col = 'category', trendline = 'lowess',  
5                   labels = {'year': 'Year', 'age': 'Age', 'category': 'Category'})  
6 fig.update_layout(showlegend = False, title_x = 0.5, margin_t = 100)  
7 fig.show()
```

## Age of Nobel Winners by Prize Category



# Nobel Prizes Data (with plotly ...)

## Age of Winners Across Fields: Further Customization

- Final adjustments on style sheet and line, marker size:

```
1 fig = px.scatter(nobel, x = 'year', y = 'age', width = 1000, height = 600,
2                  title = 'Age of Nobel Winners by Prize Category',
3                  hover_name = "fullName", color = 'category',
4                  facet_col = 'category', trendline = 'lowess',
5                  labels = {'year': '', 'age': 'Age', 'category': 'Category'})
6 fig.update_layout(showlegend = False, title_x = 0.5, margin_t = 100,
7                  title_font_size = 20, xaxis3_title = "Year")
8 fig.update_traces(marker_size = 8, marker_opacity = 0.6, line_width = 3)
9 # Remove the 'Category=' part from facet titles
10 fig.for_each_annotation(lambda a: a.update(text = a.text.split("=")[-1]))
11 fig.show()
```

## Age of Nobel Winners by Prize Category



# Summary

The matplotlib logo, featuring the word "matplotlib" in a blue sans-serif font, with a small circular icon containing a colorful pie chart.The plotly logo, featuring the word "plotly" in a blue sans-serif font, preceded by a small icon of three vertical bars of increasing height, each with a colored dot (pink, blue, and green) above it.

We've made static visualisations with `matplotlib`, `seaborn`, and `plotnine`.

- Reliable and reproducible.
- Suitable for reports, slides, and publications.

`plotly` creates interactive visualisations that allow viewers to explore data dynamically.

- Storytelling for non-technical audiences.
- Suitable for dashboards and web applications.